79-张飞单片机 2

大家晚上好，欢迎参加张飞实战电子 FOC 第 11 期学习。
那我们现在就接着上一节，再了解一下张飞单片机的 PWM 模块。

11.1 概述

ZFM32F03xx6x 可提供独立事件驱动的 PWM（脉冲宽度调制）。PWM 模块可配置成 3 对互补输出，6 个独立输出或互补和独立混合的 PWM 信号输出（如 2 对互补输出，2 个独立输出）。同时支持 0~100%占空比的边沿对齐和中心对齐模式。

PWM 模块使用 16 位计数器，它的精度在边沿对齐时是一个时钟周期，中心对齐时是 2 个时钟周期。时钟周期由 PWM 时钟源 PWM_PCLK(系统时钟)、预分频寄存器以及模数值决定。

当 PWM 信号配置成互补输出时，PWM 具有自动死区插入功能。每一个 PWM 输出可被 PWM 发生器、系统定时器、ADC 转换结果、GPIO 输入以及软件控制。非对称的 PWM 还可以允许在每半个周期改变 PWM 占空比而不需要软件参与。

大家看下，这是 PWM 模块的概述。大家打开用户手册的第 11 章可以看到这个 PWM 模块的介绍。PWM 模块可以配置为 3 对互补或者 6 路独立，或互补跟独立混合产生 PWM 的方式，支持中心对齐，边沿对齐。一般能够做电机控制的单片机，都需要有这样的功能。

11.1.1 特点

- 独立的脉宽调制模块
- 工作在系统时钟（SystemClock）
- 多达 6 路 PWM 信号输出
  - 单路独立输出模式
  - 互补输出模式
  - 混合输出模式
- 互补输出功能
  - 单独的死区上升沿/下降沿插入
  - 单独的高低脉冲宽度软件补偿
  - 中心对齐下的非对称 PWM 输出
  - 单独的高低脉冲输出极性控制
- 边沿对齐 PWM 输出

这个呢，是 PWM 模块的特点。

- 半周期重载机制
- 完善的 1 到 16 的重载频率控制
- 独立、可软件控制的 PWM 输出
- 1 路故障输入保护
- 输出极性控制
- PWM 控制寄存器写保护功能

- 可配置的 PWM 互补输出信号源
  - PWM 信号发生器
  - 外部 GPIO 管脚
  - 定时器
  - ADC 转换值

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

PWM 模块的功能越强大，那么在电机控制、或者数字电源上就越有优点。这些特点，大家了解一下，很多功能呢，跟 microchip 的单片机是相似的。

问：PWM 的分辨率最低要求有没有？

答：PWM 的分辨率越高越好，不同单片机可能这个分辨率是不一样的，分辨率越高就意味着控制精度越高。一般没有最低要求，取决于实际应用，有的要求电机能转就行，谈不上 PWM 的分辨率要求。

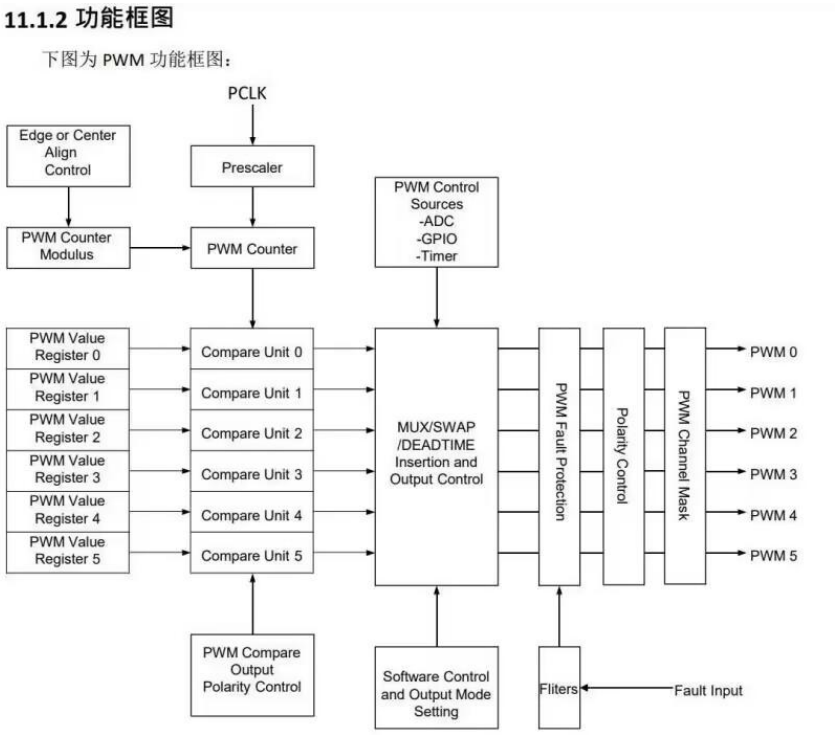


图 11-1 PWM 功能框图

这个呢，是 PWM 模块的框图，分为几个部分，我们介绍一下哈。

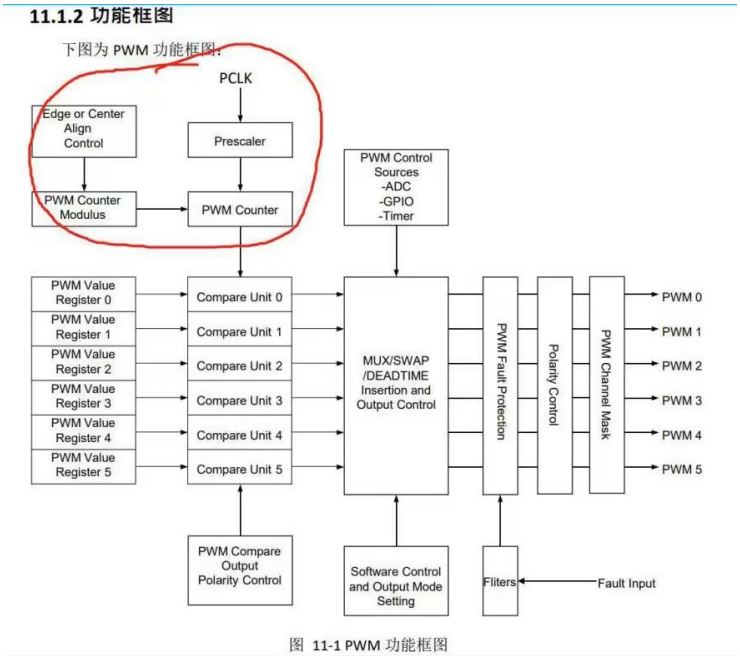


图 11-1 PWM 功能框图

这个红圈的地方，是 PWM 模块的时钟部分。它可以支持，中心对齐跟边沿对齐两种。对于边沿对齐计数模式，它的计数方向是向上计数的。而对于中心对齐计数模式，它的计数方

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

向可以支持配置，可以选择是先向上计数，还是先向下计数。当然，功能强大的一些计数器，不管是边沿对齐还是中心对齐，它的计数方向都是可以配置的。咱们这里呢，只是中心对齐模式，支持初始计数方向的配置。

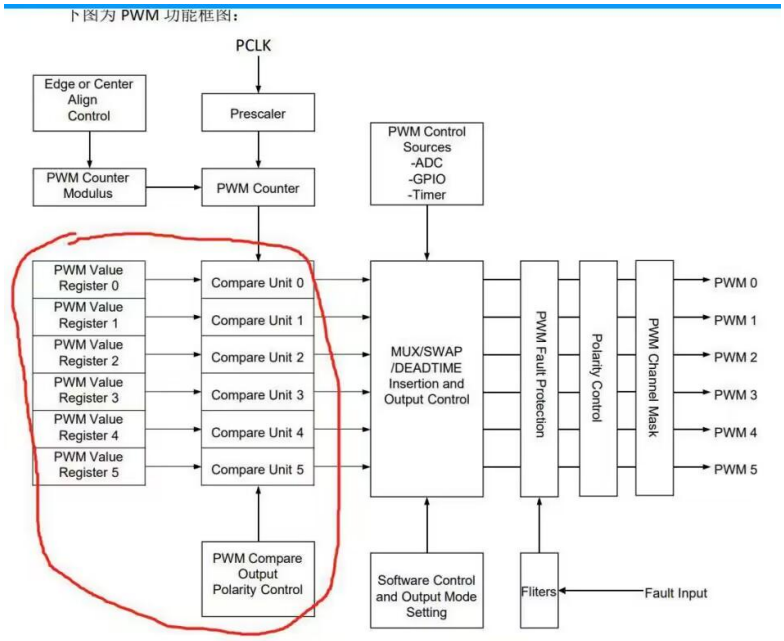


图 11-1 PWM 功能框图

这个部分是 PWM 模块产生 PWM 的重要部分，也就是用设定的比较值跟计数器值去比较。PWM 模块内部还有输出极性控制。

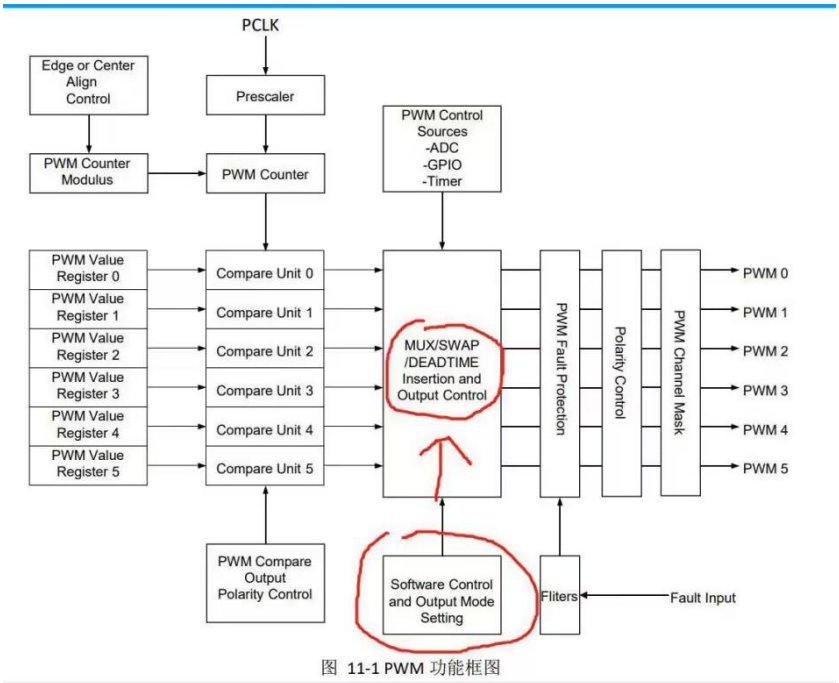


图 11-1 PWM 功能框图

然后，这个部分是实现一些死区插入，引脚交换的控制。或者，通过软件人为的改写 PWM 引脚的输出。也就是通过软件的配置，可以人为的干预前面比较模块的结果。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

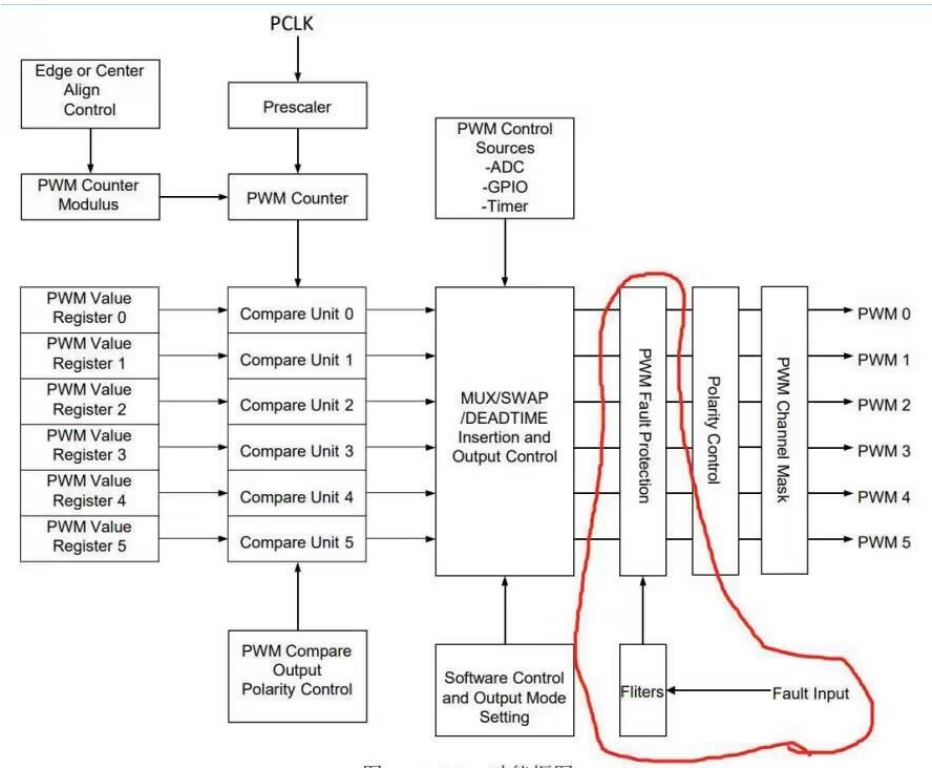


图 11-1 PWM 功能框图

再往前的话，就是故障保护的功能。它可以再往前一步来控制 PWM 模块的输出。如果有故障，它可以及时关断模块的输出。

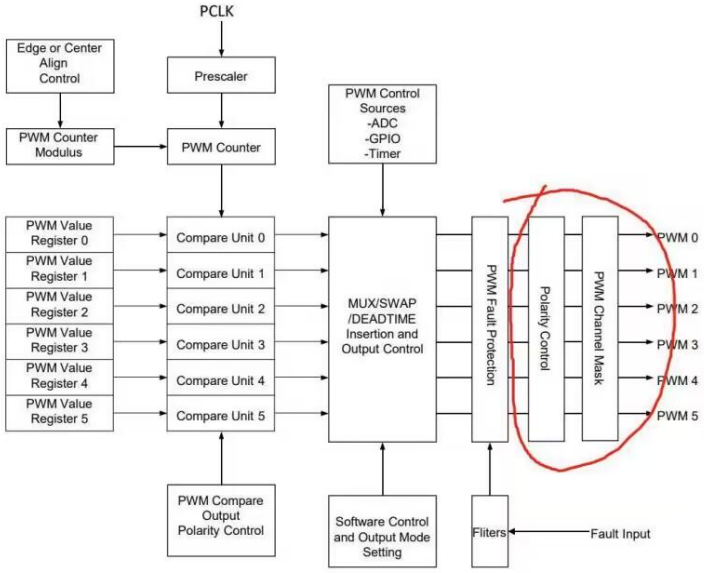


图 11-1 PWM 功能框图

最后两个是输出极性的控制跟通道是否被屏蔽输出。最终呢，信号就到了 PWM 模块的引脚上了。这就是张飞单片机的 PWM 模块的功能框图。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

表 11-1 PWM 寄存器一览(基地址 0x4004 C000)

名称	读写	偏移地址	描述	初始值
CTRL	rw	0x000	控制寄存器	0x00
FCTRL	rw	0x004	故障输入控制寄存器	0x00
FLTACK	rw	0x008	故障输入状态寄存器	0x00
OUTCTRL	rw	0x00C	输出控制寄存器	0x00
CNTR	r	0x010	计数器寄存器	0x00
CMOD	rw	0x014	模数计数器寄存器	0x00
VAL0	rw	0x018	数值寄存器 0	0x00
VAL1	rw	0x01C	数值寄存器 1	0x00
VAL2	rw	0x020	数值寄存器 2	0x00
VAL3	rw	0x024	数值寄存器 3	0x00
VAL4	rw	0x028	数值寄存器 4	0x00
VAL5	rw	0x02C	数值寄存器 5	0x00
DTIM0	rw	0x038	死区寄存器 0	0xFFFF
DTIM1	rw	0x03C	死区寄存器 1	0xFFFF
DMAP0	rw	0x040	映射失效控制寄存器 0	0xFFFF FF07
DMAP1	rw	0x044	映射失效控制寄存器 1	0x0000 FFFF
CNFG	rw	0x048	配置寄存器	0x00
CCTRL	rw	0x04C	输出通道控制寄存器	0x00
PORT	rw	0x050	端口控制寄存器	0x00
ICCTRL	rw	0x054	内部校正控制寄存器	0x00
PSCR	rw	0x060	极性反转控制寄存器	0x00
CNTRINI	w	0x064	计数器初始化寄存器	0x00

这个 PWM 模块总共有 22 个工作寄存器。当然了，这些寄存器呢， 也并不是所有的我们都要用。跟实现我们功能的相关的，我们去配置它就行。

```
/*=========================================================*/
static void PwmInitialize(void)
{
    IOCFG -> PIOA[7] = 0x00000021;
    IOCFG -> PIOA[8] = 0x00000021;
    IOCFG -> PIOA[9] = 0x00000021;
    IOCFG -> PIOA[10] = 0x00000021;
    IOCFG -> PIOB[0] = 0x00000021;
    IOCFG -> PIOB[1] = 0x00000021;

    PWM -> CTRL = 0x00800000; //PwmCounter = CMOD Trigger Signal Out
    PWM -> CMOD = PWM_PERIOD_VALUE >> 1; //48MHz/10KHz = 4800 Center Align CMOD = 4800 / 2 - 1 = 2399

    PWM -> VAL0 = PWM_PERIOD_VALUE >> 3; //10%
    PWM -> VAL2 = PWM_PERIOD_VALUE >> 3;
    PWM -> VAL4 = PWM_PERIOD_VALUE >> 3;

    PWM -> DTIM0 = 96; //Deadtime 2us
    PWM -> DTIM1 = 96;
    PWM -> CNFG = 0; //Center Align Complementary Output
    PWM -> ICCTRL = 0x00000010; //PIN OUT ENABLE
    PWM -> PSCR = 0x00000000; //timer counter > VAL low level
    PWM -> CNTRINI = 0x00000000; //PWM timer INIT VALUE :0

    PWM -> CTRL &= 0xfffffffd;
    PWM -> CTRL |= 0x00000002; //LDOK = 1

    PWM -> CTRL |= 0x00000001; //PWMEN = 1
    PWM -> CTRL |= 0x01000000; //OUT0EN = 1 Enable Can Enter Adc Int
}
```

这是 PWM 模块寄存器配置的代码 。

```
/*=========================================================*/
static void PwmInitialize(void)
{
    IOCFG -> PIOA[7] = 0x00000021;
    IOCFG -> PIOA[8] = 0x00000021;
    IOCFG -> PIOA[9] = 0x00000021;
    IOCFG -> PIOA[10] = 0x00000021;
    IOCFG -> PIOB[0] = 0x00000021;
    IOCFG -> PIOB[1] = 0x00000021;

    PWM -> CTRL = 0x00800000; //PwmCounter = CMOD Trigger Signal Out
    PWM -> CMOD = PWM_PERIOD_VALUE >> 1; //48MHz/10KHz = 4800 Center Align CMOD = 4800 / 2 - 1 = 2399

    PWM -> VAL0 = PWM_PERIOD_VALUE >> 3; //10%
    PWM -> VAL2 = PWM_PERIOD_VALUE >> 3;
    PWM -> VAL4 = PWM_PERIOD_VALUE >> 3;

    PWM -> DTIM0 = 96; //Deadtime 2us
    PWM -> DTIM1 = 96;
    PWM -> CNFG = 0; //Center Align Complementary Output
    PWM -> ICCTRL = 0x00000010; //PIN OUT ENABLE
    PWM -> PSCR = 0x00000000; //timer counter > VAL low level
    PWM -> CNTRINI = 0x00000000; //PWM timer INIT VALUE :0

    PWM -> CTRL &= 0xfffffffd;
    PWM -> CTRL |= 0x00000002; //LDOK = 1

    PWM -> CTRL |= 0x00000001; //PWMEN = 1
    PWM -> CTRL |= 0x01000000; //OUT0EN = 1 Enable Can Enter Adc Int
}
```

我们使用 PA7 PA8 PA9 PA10 PB0 PB1 这 6 个引脚作为 PWM 引脚。所以呢，我们需要设置这 6 个引脚作为 PWM 模块的外设复用引脚。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

主要呢，就是配置上图红框的 6 个寄存器。

Bits2:0 FUNC[2:0]：功能选择
写/读：
0: 选择功能 PA7
1: 选择功能 PWM_OUT1
2: 选择功能 TIM2_CAP1
3: 选择功能 TIM2_MAT1
4: 选择功能 SPI_MOSI
5: 保留
6: 选择功能 ADC_IN7

大家看，bit2~bit0　设置为 1 的话，就是把引脚设置为 PWM 外设功能。

1: 已使能开漏模式。
注：这不是真正的开漏模式。输入不能上拉至超过 VDD(IO)
Bit9 DRV：驱动电流
写/读：
0: 选择高电流模式
1: 选择低电流模式
Bit8 ADM：模拟/数字功能选择

张飞电子 www.zhangfeidz.com
71
ZFM32F030 系列 32 位微控制器用户手册 扫码索取样片：

写/读：
0: 选择数字功能
1: 选择模拟功能

然后，这个引脚的驱动能力需要注意一下。因为 PWM 是作为一个驱动信号使用的，所以最好引脚的驱动电流呢，我们可以设置为高电流模式。也就是把 bit9 设置为 0，　这是关于引脚的功能配置。引脚配置好之后，接着就可以配置 PWM 模块的寄存器。

```
/*=========================================================*/
static void PwmInitialize(void)
{
    IOCFG -> PIOA[7] = 0x00000021;
    IOCFG -> PIOA[8] = 0x00000021;
    IOCFG -> PIOA[9] = 0x00000021;
    IOCFG -> PIOA[10] = 0x00000021;
    IOCFG -> PIOB[0] = 0x00000021;
    IOCFG -> PIOB[1] = 0x00000021;

    PWM -> CTRL = 0x00800000; //PwmCounter = CMOD Trigger Signal Out
    PWM -> CMOD = PWM_PERIOD_VALUE >> 1; //48MHz/10KHz = 4800 Center Align CMOD = 4800 / 2 - 1 = 2399

    PWM -> VAL0 = PWM_PERIOD_VALUE >> 3; //10%
    PWM -> VAL2 = PWM_PERIOD_VALUE >> 3;
    PWM -> VAL4 = PWM_PERIOD_VALUE >> 3;

    PWM -> DTIM0 = 96; //Deadtime 2us
    PWM -> DTIM1 = 96;
    PWM -> CNFG = 0; //Center Align Complementary Output
    PWM -> ICCTRL = 0x00000010; //PIN OUT ENABLE
    PWM -> PSCR = 0x00000000; //timer counter > VAL low level
    PWM -> CNTRINI = 0x00000000; //PWM timer INIT VALUE :0

    PWM -> CTRL &= 0xfffffffd;
    PWM -> CTRL |= 0x00000002; //LDOK = 1

    PWM -> CTRL |= 0x00000001; //PWMEN = 1
    PWM -> CTRL |= 0x01000000; //OUT0EN = 1 Enable Can Enter Adc Int
}
```

首先呢，我们设置触发 ADC 采样的触发点。之前 MICROCHIP 讲过的触发点设置，我们是设置在下桥开通的中点时刻去采样的。而我们张飞单片机的话，中心对齐模式计数器，它是先向上计数，然后再向下计数。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

所以呢，下桥开通的中点时刻，就是当计数器计数值等于 CMOD（周期设定值）的时候。也就是说，我们设定的触发点，就是设置当计数器等于 CMOD 的时候，产生 ADC 触发信号。

```
/*============================================================*/
static void PwmInitialize(void)
{
    IOCFG -> PIOA[7] = 0x00000021;
    IOCFG -> PIOA[8] = 0x00000021;
    IOCFG -> PIOA[9] = 0x00000021;
    IOCFG -> PIOA[10] = 0x00000021;
    IOCFG -> PIOB[0] = 0x00000021;
    IOCFG -> PIOB[1] = 0x00000021;

    PWM -> CTRL = 0x00800000; //PwmCounter = CMOD Trigger Signal Out
    PWM -> CMOD = PWM_PERIOD_VALUE >> 1; //48MHz/10KHz = 4800 Center Align CMOD = 4800 / 2 - 1 = 2399

    PWM -> VAL0 = PWM_PERIOD_VALUE >> 3; //10%
    PWM -> VAL2 = PWM_PERIOD_VALUE >> 3;
    PWM -> VAL4 = PWM_PERIOD_VALUE >> 3;

    PWM -> DTIM0 = 96; //Deadtime 2us
    PWM -> DTIM1 = 96;
    PWM -> CNFG = 0; //Center Align Complementary Output
    PWM -> ICCTRL = 0x00000010; //PIN OUT ENABLE
    PWM -> PSCR = 0x00000000; //timer counter > VAL low level
    PWM -> CNTRINI = 0x00000000; //PWM timer INIT VALUE :0

    PWM -> CTRL &= 0xfffffffd;
    PWM -> CTRL |= 0x00000002; //LDOK = 1

    PWM -> CTRL |= 0x00000001; //PWMEN = 1
    PWM -> CTRL |= 0x01000000; //OUT0EN = 1 Enable Can Enter Adc Int
}
```

然后呢，接着就是设置周期寄存器、比较值寄存器的初值。这个 CMOD 的话，就是决定 PWM 波的频率。上面注释举了一个例子，如果我们要设置 PWM 频率是 10KHz 的话，对应的 CMOD 的设定值就是按照上述注释计算。

```
PWM -> CTRL = 0x00800000; //PwmCounter = CMOD Trigger Signal Out
PWM -> CMOD = PWM_PERIOD_VALUE >> 1; //48MHz/10KHz = 4800 Center Align CMOD = 4800 / 2 - 1 = 2399

PWM -> VAL0 = PWM_PERIOD_VALUE >> 3; //10%
PWM -> VAL2 = PWM_PERIOD_VALUE >> 3;
PWM -> VAL4 = PWM_PERIOD_VALUE >> 3;

PWM -> DTIM0 = 96; //Deadtime 2us
PWM -> DTIM1 = 96;
PWM -> CNFG = 0; //Center Align Complementary Output
PWM -> ICCTRL = 0x00000010; //PIN OUT ENABLE
PWM -> PSCR = 0x00000000; //timer counter > VAL low level
PWM -> CNTRINI = 0x00000000; //PWM timer INIT VALUE :0

PWM -> CTRL &= 0xfffffffd;
PWM -> CTRL |= 0x00000002; //LDOK = 1

PWM -> CTRL |= 0x00000001; //PWMEN = 1
PWM -> CTRL |= 0x01000000; //OUT0EN = 1 Enable Can Enter Adc Int
```

接着呢，是设置死区时间， 96 的话，对应的就是 2us 的死区。因为我们单片机的主频是 48MHz，1/48MHz 对应一个时间，乘以 48 的话就是 1us，96 的话就是 2us 了。

```
PWM -> DTIM0 = 96; //Deadtime 2us
PWM -> DTIM1 = 96;
PWM -> CNFG = 0; //Center Align Complementary Output
PWM -> ICCTRL = 0x00000010; //PIN OUT ENABLE
PWM -> PSCR = 0x00000000; //timer counter > VAL low level
PWM -> CNTRINI = 0x00000000; //PWM timer INIT VALUE :0

PWM -> CTRL &= 0xfffffffd;
PWM -> CTRL |= 0x00000002; //LDOK = 1

PWM -> CTRL |= 0x00000001; //PWMEN = 1
PWM -> CTRL |= 0x01000000; //OUT0EN = 1 Enable Can Enter Adc In
```

然后，这个寄存器是设置 PWM 的模式，这里设置的是中心对齐的互补 PWM 模式。对应的 CNFG 寄存器的值呢，也就是 0。

问：这里 pwm 是 10k 是么？

答：是的。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
PWM -> VAL0 = PWM_PERIOD_VALUE >> 3; //10%
PWM -> VAL2 = PWM_PERIOD_VALUE >> 3;
PWM -> VAL4 = PWM_PERIOD_VALUE >> 3;

PWM -> DTIM0 = 96; //Deadtime 2us
PWM -> DTIM1 = 96;
PWM -> CNFG = 0; //Center Align Complementary Output
PWM -> ICCTRL = 0x00000010; //PIN OUT ENABLE
PWM -> PSCR = 0x00000000; //timer counter > VAL low level
PWM -> CNTRINI = 0x00000000; //PWM timer INIT VALUE :0

PWM -> CTRL &= 0xfffffffd;
PWM -> CTRL |= 0x00000002; //LDOK = 1

PWM -> CTRL |= 0x00000001; //PWMEN = 1
PWM -> CTRL |= 0x01000000; //OUT0EN = 1 Enable Can Enter Adc Int
```

然后呢，这个寄存器的话，主要是把引脚输出 PWM 的功能打开， 否则的话，输出不了 PWM 波。

```
PWM -> CTRL = 0x00800000; //PwmCounter = CMOD Trigger Signal Out
PWM -> CMOD = PWM_PERIOD_VALUE >> 1; //48MHz/10KHz = 4800 Center Align CMOD = 4800 / 2 - 1 = 2399

PWM -> VAL0 = PWM_PERIOD_VALUE >> 3; //10%
PWM -> VAL2 = PWM_PERIOD_VALUE >> 3;
PWM -> VAL4 = PWM_PERIOD_VALUE >> 3;

PWM -> DTIM0 = 96; //Deadtime 2us
PWM -> DTIM1 = 96;
PWM -> CNFG = 0; //Center Align Complementary Output
PWM -> ICCTRL = 0x00000010; //PIN OUT ENABLE
PWM -> PSCR = 0x00000000; //timer counter > VAL low level
PWM -> CNTRINI = 0x00000000; //PWM timer INIT VALUE :0

PWM -> CTRL &= 0xfffffffd;
PWM -> CTRL |= 0x00000002; //LDOK = 1

PWM -> CTRL |= 0x00000001; //PWMEN = 1
PWM -> CTRL |= 0x01000000; //OUT0EN = 1 Enable Can Enter Adc Int
```

接着，就是设置比较输出的极性跟计数器的初值。这里呢，设置值是 0，实际上如果默认值是 0 的话，这里也可以不用写，但是写出来的话，更清晰一些。

```
PWM -> CTRL = 0x00800000; //PwmCounter = CMOD Trigger Signal Out
PWM -> CMOD = PWM_PERIOD_VALUE >> 1; //48MHz/10KHz = 4800 Center Align CMOD = 4800 / 2 - 1 = 2399

PWM -> VAL0 = PWM_PERIOD_VALUE >> 3; //10%
PWM -> VAL2 = PWM_PERIOD_VALUE >> 3;
PWM -> VAL4 = PWM_PERIOD_VALUE >> 3;

PWM -> DTIM0 = 96; //Deadtime 2us
PWM -> DTIM1 = 96;
PWM -> CNFG = 0; //Center Align Complementary Output
PWM -> ICCTRL = 0x00000010; //PIN OUT ENABLE
PWM -> PSCR = 0x00000000; //timer counter > VAL low level
PWM -> CNTRINI = 0x00000000; //PWM timer INIT VALUE :0

PWM -> CTRL &= 0xfffffffd;
PWM -> CTRL |= 0x00000002; //LDOK = 1

PWM -> CTRL |= 0x00000001; //PWMEN = 1
PWM -> CTRL |= 0x01000000; //OUT0EN = 1 Enable Can Enter Adc Int
```

然后呢，要装载寄存器，需要把 LDOK 位设置为 1。

```
PWM -> DTIM0 = 96; //Deadtime 2us
PWM -> DTIM1 = 96;
PWM -> CNFG = 0; //Center Align Complementary Output
PWM -> ICCTRL = 0x00000010; //PIN OUT ENABLE
PWM -> PSCR = 0x00000000; //timer counter > VAL low level
PWM -> CNTRINI = 0x00000000; //PWM timer INIT VALUE :0

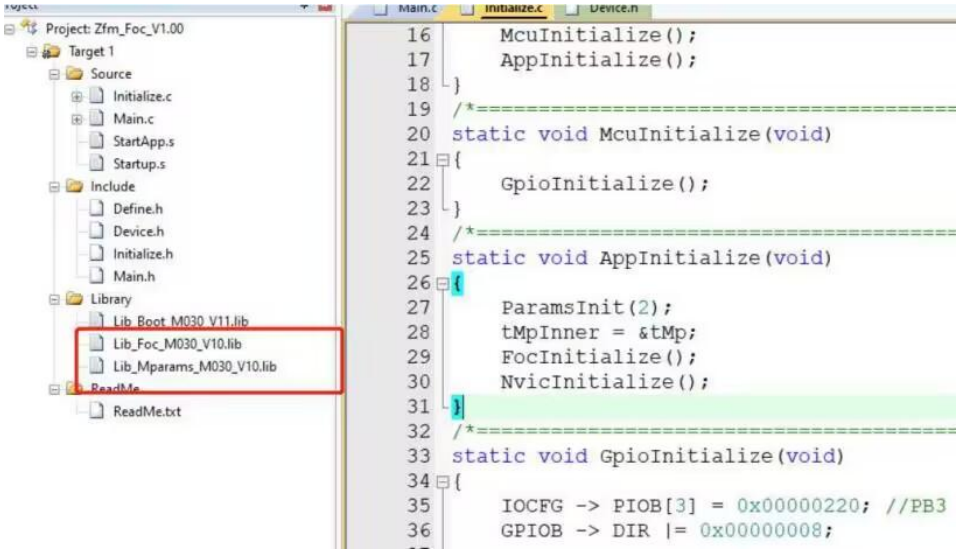
PWM -> CTRL &= 0xfffffffd;
PWM -> CTRL |= 0x00000002; //LDOK = 1

PWM -> CTRL |= 0x00000001; //PWMEN = 1
PWM -> CTRL |= 0x01000000; //OUT0EN = 1 Enable Can Enter Adc Int
*============================================================*/
```

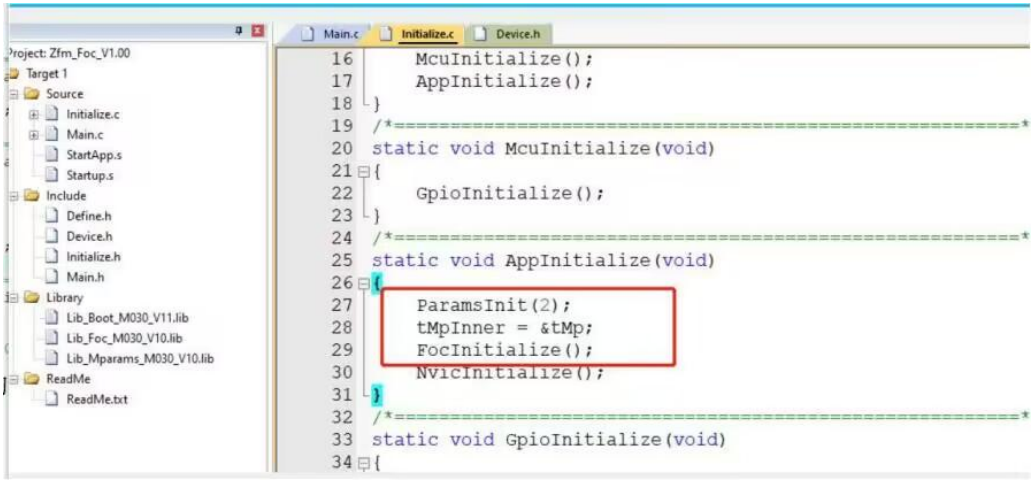

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

最后呢，把 PWM 模块使能，以及通道使能，这样的话，PWM 模块就配置好了。虽然说，配置寄存器稍微复杂一些，但是如果第一次配置好之后，后面功能有简单变化的话，在这个基础上去修改寄存器的值，也是一劳永逸的事情。

ADC 跟 PWM 模块分析完之后，下面我们来看下对应的 FOC 库怎么调用？



如果我们要使用张飞单片机让电机转动的话，需要包含左边的两个库。我们使用库的话，要做的就是对库中需要的函数进行一下调用。



第 27 行这个函数，是关于参数设置的初始化函数，它是来自于库 Lib_Mparams_M030_V10.lib。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
/*==========================================================
描述          : 张飞单片机无感FOC测试例程
项目          : Zfm_Foc_V1.00
单片机型号    : ZFM32F030K6U
日期          : 2023.06.01
文件          : Initialize.h
功能          : 初始化头文件
作者          : 张飞实战电子
==========================================================*/
#include "..\Include\Device.h"
#include "..\Include\Define.h"
/*==========================================================*/
void Initialize(void);

static void McuInitialize(void);
static void AppInitialize(void);
static void GpioInitialize(void);
//static void AdcInitialize(void);
//static void PwmInitialize(void);
static void NvicInitialize(void);

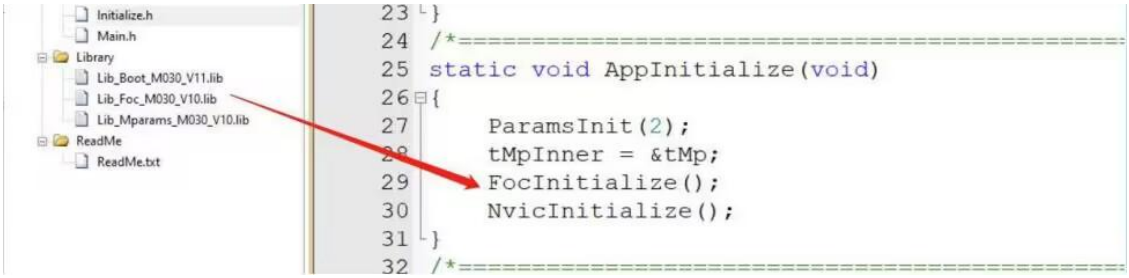
extern void FocInitialize(void);
extern void LedKeyInit(void);
extern void ParamsInit(unsigned char UartX);

extern tMp_Type tMp __attribute__((section("NOT_INIT"),zero_init));
extern tMp_Type *tMpInner __attribute__((section("NOT_INIT"),zero_init));
/*=========================Copyright belongs to ZhangFeiShiZhanDianZi=======
```

使用它的话，主要是在初始化的头文件中声明一下函数。这个函数的话，带有一个形参，它表示串口号，这里的 2，表示使用的是 Uart2，这个串口功能呢，是为了上位机修改参数的然后呢，下面的变量声明，是进行一个参数的传递。

```
/*==========================================================
static void AppInitialize(void)
{
    ParamsInit(2);
    tMpInner = &tMp;
    FocInitialize();
    NvicInitialize();
}
/*==========================================================
```

也就是说，关于参数的库文件，我们需要增加的初始化配置就是上图红框的这两句话。



然后，是 FOC 控制库的初始化。这个函数里面呢，其实包含我们前面跟大家分析的 ADC 模块跟 PWM 模块的操作。配置方式呢，跟我们前面分析的代码一样。只不过呢，这里调用库文件的话，库文件中写了，外面就不写了。这样的话，关于使用库文件，所需要的初始化就完成了。然后，我们要操控电机转动的话，需要再使用 Lib_Foc_M030_V10.lib 中的中断跟应用程序。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。



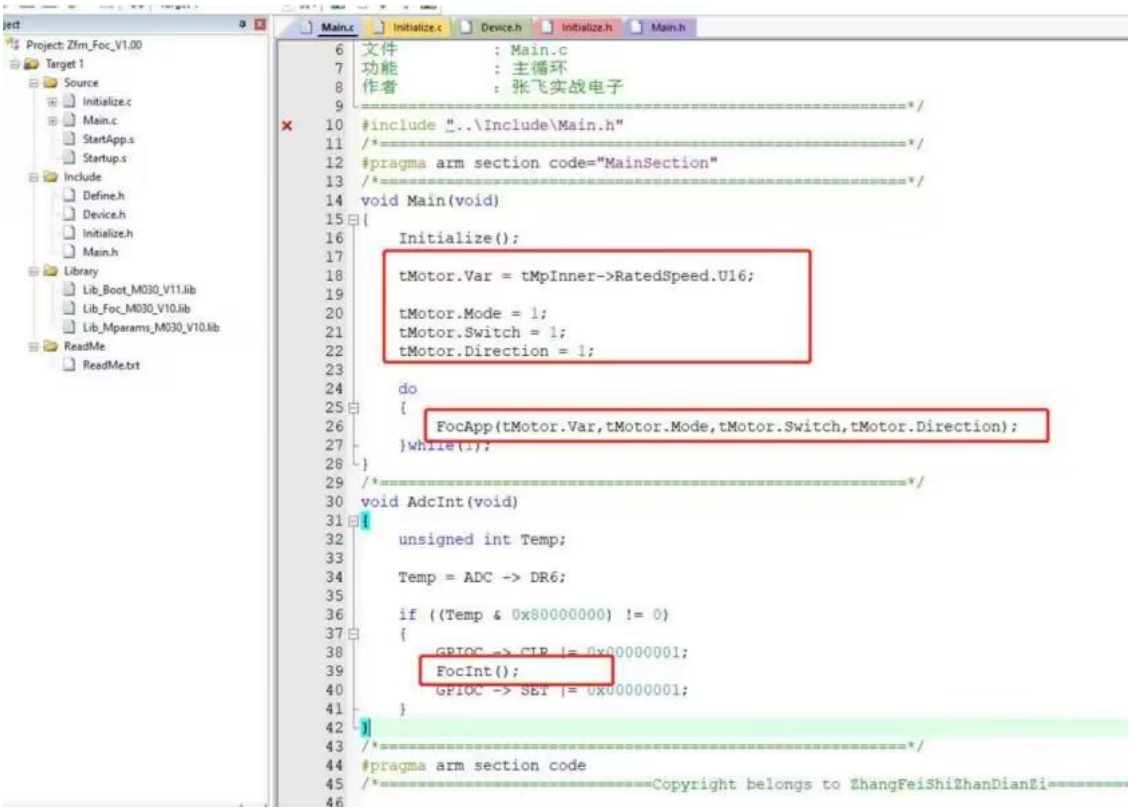
```
/*=============================================================
描述          : 张飞单片机无感FOC测试例程
项目          : Zfm_Foc_V1.00
单片机型号    : ZFM32F030K6U
日期          : 2023.06.01
文件          : Main.h
功能          : 主程序头文件
作者          : 张飞实战电子
=============================================================*/
#include "..\Include\Device.h"
#include "..\Include\Define.h"
/*=============================================================*/
void Main(void);
void Delay(void);

static void SoftWareCounter(void);

extern void Initialize(void);
extern void FocInt(void);
extern void FocApp(unsigned short X, unsigned char M, unsigned char S, unsigned char D);
/*=============================================================*/
typedef struct
{
    unsigned short Var;
    unsigned char Mode;
    unsigned char Switch;
    unsigned char Direction;
}tMotor_Type;
tMotor_Type tMotor __attribute__((section("NOT_INIT"),zero_init));

extern tMp_Type *tMpInner __attribute__((section("NOT_INIT"),zero_init));
/*=============================Copyright belongs to ZhangFeiShiZhanDianZi
```

这里呢，定义了一个结构体来进行电机转动的参数设置。可以设置转速、模式、运行停止、旋转方向等。



```
文件          : Main.c
功能          : 主循环
作者          : 张飞实战电子
=============================================================*/
#include "..\Include\Main.h"
/*=============================================================*/
#pragma arm section code="MainSection"
/*=============================================================*/
void Main(void)
{
    Initialize();

    tMotor.Var = tMpInner->RatedSpeed.U16;

    tMotor.Mode = 1;
    tMotor.Switch = 1;
    tMotor.Direction = 1;

    do
    {
        FocApp(tMotor.Var,tMotor.Mode,tMotor.Switch,tMotor.Direction);
    }while(1);
}
/*=============================================================*/
void AdcInt(void)
{
    unsigned int Temp;

    Temp = ADC -> DR6;

    if ((Temp & 0x80000000) != 0)
    {
        GPIOC -> CLR |= 0x00000001;
        FocInt();
        GPIOC -> SET |= 0x00000001;
    }
}
/*=============================================================*/
#pragma arm section code
/*=============================Copyright belongs to ZhangFeiShiZhanDianZi
```

然后，这样去调用一下，FOC 库就成功使用了。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
/*=========================================================*/
void Main(void)
{
    Initialize();

    tMotor.Var = tMpInner->RatedSpeed.U16;
    tMotor.Mode = 1;
    tMotor.Switch = 1;
    tMotor.Direction = 1;

    do
    {
        FocApp(tMotor.Var,tMotor.Mode,tMotor.Switch,tMotor.Direction);
    }while(1);
}
/*=========================================================*/
```

转速
速度闭环
使能转动
逆时针

这几个参数的话，分别表示上述含义。

```
/*=========================================================*/
void AdcInt(void)
{
    unsigned int Temp;

    Temp = ADC -> DR6;

    if ((Temp & 0x80000000) != 0)
    {
        GPIOC -> CLR |= 0x00000001;
        FocInt();
        GPIOC -> SET |= 0x00000001;
    }
}
/*=========================================================*/
```

然后，这个 FocInt 函数的话，就是在 ADC 中断中调用的。这样的话，就可以控制电机运转了。所以，关于库文件的使用，方法也是比较简单的，上传的文件里面包含了库文件。刚才呢，我们也介绍到了参数库文件。

Main.c | Initialize.c | Device.h | Initialize.h | Main.h | Startup.s | StartApp.s | Define.h

```
        unsigned char Low;
        unsigned char High;
    } Bytes;
} UNION_U16;
/*=========================================================*/
typedef struct
{
    UNION_U16 Inductance;              //0
    UNION_U16 Resistance;              //1
    UNION_U16 RotorPoles;              //2
    UNION_U16 StartCurrent;            //3
    UNION_U16 PeakCurrent;             //4
    UNION_U16 RatedCurrent;            //5
    UNION_U16 ProtectPower;            //6
    UNION_U16 PeakPower;               //7

    UNION_U16 RatedVoltage;            //8
    UNION_U16 RatedSpeed;              //9
    UNION_U16 MaximumSpeed;            //10
    UNION_U16 MinimumSpeed;            //11
    UNION_U16 MonitorSpeed;            //12
    UNION_U16 MaximumVoltage;          //13
    UNION_U16 MinimumVoltage;          //14
    UNION_U16 BusVoltageFactor;        //15
    UNION_U16 BusCurrentFactor;        //16

    UNION_U16 PhaseCurrentFactor;      //17
    UNION_U16 OverTempValue;           //18
    UNION_U16 StabilizingFactor;       //19
    UNION_U16 DeadTimeValue;           //20
    UNION_U16 MinimumDutyPercent;      //22
    UNION_U16 SwitchDutyPercent;       //23
    UNION_U16 StartDutyPercent;        //24

    UNION_U16 StartTimeValue;          //25
    UNION_U16 CurrentKpValue;          //26
    UNION_U16 CurrentKiValue;          //27
    UNION_U16 SpeedKpValue;            //28
    UNION_U16 SpeedKiValue;            //29
    UNION_U16 SpeedPeriodValue;        //30
    UNION_U16 AddSpeedValue;           //31
    UNION_U16 DecSpeedValue;           //32

    UNION_U16 Status;                  //33
    unsigned char UartControlEnable;
    unsigned char DataObserveEnable;   //34
    unsigned char DebugOutputEnable;
    unsigned char SpeedLoopEnable;     //35
}tMp_Type;
//==========Copyright belongs to ZhangFeiShiZhanDianZi==========
```


本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

上面这结构体就是单片机传递用于设置参数的结构体。



如果说，我们想要修改这些参数，可以使用配套的参数配置上位机来修改。设置好之后呢，这些参数会保存到 Flash 中。这样的话，针对不同的电机，可以去修改对应的参数，来适配自己的电机。那么，到目前为止，关于我们课程的内容，就全部结束了。

问：咱们这个中断程序时间不要要超过 pwm 周期的百分之多少？
答：80%~90%。

问：电机运行，程序调试的好不好，有哪些经验参数指标建议么？
答：观测电流波形很重要，控制不好，电流波形会异常，其次是效率，效率高是大多数控制追求的目标。

我们课程呢，仅是 FOC 电机控制的基础入门课程。大家在实际工作中，再不同的产品应用中，可能还会遇到各种各样的问题。不同的产品呢，对电机驱动的关注点会有所不同。例如，有的关注低速，有的关注高速。有的呢，关心带载能力，有的关注效率。所以啊，针对不同的应用，仍然还有很多种不同的探索策略跟方案。

而且呢，我们课程呢，仅涉及到了电流环跟速度环。实际上呢，更高级的还会有伺服控制，也就是需要位置环的参与。伺服控制呢，它对精度和实时性的要求会更高。要想做伺服的话，一般的单片机可能搞不定，需要性能更强的单片机，或者 DSP，甚至是 FPGA 。也就是说，更高级的控制，需要高精度、高实时性，所以控制都会往硬件化的方向去发展。这些方向呢，大家也可以选择一个去深入。

希望大家通过本次培训，把基础打好，然后找到属于自己的方向，继续在电机驱动领域前行。在这里呢，也祝大家在今后的工作及生活中，越来越好，希望大家都能够升职、加薪!!也感谢大家这段时间的支持。

大家呢，后续可以继续抽时间进行复习，有疑问的话，可以继续交流讨论，优先找助教。那我们今天就到这里了，谢谢大家，晚安！江湖见。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。